Class 5: Code Walkthrough

Building & Training a Modern LLM — From Scratch

RMSNorm · RoPE · GQA · SwiGLU

SECTION 0

Setup & Data

Loading Shakespeare + Character Tokenizer

Character Tokenizer

```
chars = sorted(set(text))          # All unique chars
vocab_size = len(chars)            # 65

char_to_idx = {c: i for i, c in enumerate(chars)}
idx_to_char = {i: c for i, c in enumerate(chars)}

def encode(s):
    return [char_to_idx[c] for c in s]

def decode(ids):
    return "".join([idx_to_char[i] for i in ids])
```

- Get every unique character from Shakespeare → 65 total
- Two lookup dictionaries: char→int and int→char
- encode() converts string to list of integers
- decode() converts integers back to string

```
"Hello"  →  [20, 43, 50, 50, 53]
```

get_batch() — Grab Training Data

```
def get_batch(split, batch_size, context_length):
    d = train_data if split == "train" else val_data
    ix = torch.randint(len(d) - context_length, (batch_size,))
    x = torch.stack([d[i:i+context_length] for i in ix])
    y = torch.stack([d[i+1:i+context_length+1] for i in ix])
    return x.to(device), y.to(device)
```

- Pick random starting positions in the text
- x = input chunk, y = same chunk shifted right by 1
- Target at each position = the NEXT character
- .to(device) moves data to GPU

```
x = "To be o"  → shifted by 1 →  y = "o be or"
```

Why Shift by 1?

Pos	Given (context so far)		Predict
0	`T`	→	`o`
1	`To`	→	`␣`
2	`To␣`	→	`b`
3	`To␣b`	→	`e`
4	`To␣be`	→	`,`

One sequence of length N gives us N training examples!

SECTION 1

RMSNorm

Root Mean Square Normalization

Why RMSNorm?

LayerNorm (old)

- Subtract mean
- Divide by std
- Scale + Shift
- 4 ops, 2 learned params

RMSNorm (new)

- Divide by RMS
- Scale only
- 2 ops, 1 learned param

Simpler, faster, works just as well. Used in LLaMA, Mistral, Gemma.

RMSNorm — Code

```
class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self, x):
        rms = torch.sqrt(
            x.pow(2).mean(dim=-1, keepdim=True) + self.eps
        )
        return (x / rms) * self.weight
```

- weight — learnable scale vector (starts as all 1s)
- x.pow(2).mean() — average of squared values
- sqrt(…) — root of that mean = the RMS
- eps = 1e-6 prevents division by zero

```
[3, -1, 2, -4]  →  RMS=√7.5≈2.74 → [1.09, -0.36, 0.73, -1.46]
```

Dropout — Regularization

```
DROPOUT = 0.2
```

- During training: randomly zeroes 20% of values
- Forces model to not rely on any single feature
- During inference: all values are active
- Prevents memorization (overfitting)

SECTION 2

RoPE

Rotary Positional Embeddings

Why RoPE?

Sinusoidal (old)

- ADD fixed vector to embedding
- Encodes absolute position
- Position 5 always looks the same

RoPE (new)

- ROTATE Q and K vectors
- Dot product captures relative distance
- Only the gap between positions matters
- Applied to Q and K only

The Clock Analogy

Position 3 → Position 5

gap = 2

=

Position 7 → Position 9

gap = 2

After RoPE rotation, Q·K depends only on the RELATIVE distance between positions, not the absolute positions themselves.

Different absolute positions, SAME relative distance

RoPE — Precompute Frequencies

```
def precompute_rope_freqs(head_dim, max_seq_len, base=10000.0):
    freqs = 1.0 / (base ** (
        torch.arange(0, head_dim, 2).float() / head_dim
    ))
    positions = torch.arange(max_seq_len).float()
    angles = torch.outer(positions, freqs)
    return torch.cos(angles), torch.sin(angles)
```

- Each dimension PAIR gets a different rotation speed
- Low dims → fast rotation → short-range patterns
- High dims → slow rotation → long-range patterns
- Precomputed once, reused every forward pass

RoPE — Apply Rotation

```
def apply_rope(x, cos, sin):
    seq_len = x.shape[2]
    cos = cos[:seq_len].unsqueeze(0).unsqueeze(0)
    sin = sin[:seq_len].unsqueeze(0).unsqueeze(0)

    x1 = x[..., ::2]     # even dims
    x2 = x[..., 1::2]    # odd dims

    out1 = x1 * cos - x2 * sin
    out2 = x1 * sin + x2 * cos

    return torch.stack([out1, out2], dim=-1).flatten(-2)
```

- Split into pairs: (d0,d1), (d2,d3), (d4,d5)…
- Apply 2D rotation matrix to each pair
- After rotation, Q·K depends on RELATIVE position

The 2D Rotation Formula

2D rotation matrix applied to each dimension pair:

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 \cdot \cos - x_2 \cdot \sin \\ x_1 \cdot \sin + x_2 \cdot \cos \end{bmatrix}$$

θ depends on position → Q·K captures relative distance

- Each dimension pair (d_0,d_1), (d_2,d_3), ... gets its own rotation angle
- Low dimensions → fast rotation → captures short-range patterns
- High dimensions → slow rotation → captures long-range patterns

Each dimension pair is rotated by an angle that depends on position

SECTION 3

GQA

Grouped Query Attention

GQA — Sharing KV Heads

Standard MHA	GQA (ours)
Q0 ↔ K0, V0	Q0 ↔ K0, V0
Q1 ↔ K1, V1	Q1 ↔ K0, V0 (shared)
Q2 ↔ K2, V2	Q2 ↔ K0, V0 (shared)
Q3 ↔ K3, V3	Q3 ↔ K0, V0 (shared)
Q4 ↔ K4, V4	Q4 ↔ K1, V1
Q5 ↔ K5, V5	Q5 ↔ K1, V1 (shared)
Q6 ↔ K6, V6	Q6 ↔ K1, V1 (shared)
Q7 ↔ K7, V7	Q7 ↔ K1, V1 (shared)
8 KV heads	2 KV heads → 4× savings

KV cache is 4× smaller → much faster inference

repeat_kv() — Expand KV Heads

```
def repeat_kv(x, n_rep):
    if n_rep == 1:
        return x
    b, n_kv, seq, hd = x.shape
    return (x[:, :, None, :, :]
            .expand(b, n_kv, n_rep, seq, hd)
            .reshape(b, n_kv * n_rep, seq, hd))
```

- Copies each KV head n_rep times to match Q heads
- expand() is memory-efficient (no actual data copy)
- Heads 0-3 share KV head 0, Heads 4-7 share KV head 1

```
K: [b, 2, seq, 32]  → expand →  K: [b, 8, seq, 32]
```

GQA — Projections

```
self.q_proj = nn.Linear(d_model, n_heads * head_dim)
# 256 → 256  (8 heads × 32 dim)

self.k_proj = nn.Linear(d_model, n_kv_heads * head_dim)
# 256 → 64   (2 heads × 32 dim)  ← 4× smaller!

self.v_proj = nn.Linear(d_model, n_kv_heads * head_dim)
# 256 → 64   (2 heads × 32 dim)  ← 4× smaller!

self.o_proj = nn.Linear(n_heads * head_dim, d_model)
# 256 → 256
```

- Q projection: full size — all 8 heads need unique queries
- K and V projections: 4× smaller — only 2 heads
- This is where the memory savings come from
- All bias=False (modern convention)

GQA — Forward: Project, Reshape, RoPE

```
q = self.q_proj(x)     # [b, seq, 256]
k = self.k_proj(x)     # [b, seq, 64]
v = self.v_proj(x)     # [b, seq, 64]

q = q.view(b, seq, 8, 32).transpose(1, 2)  # [b,8,seq,32]
k = k.view(b, seq, 2, 32).transpose(1, 2)  # [b,2,seq,32]
v = v.view(b, seq, 2, 32).transpose(1, 2)  # [b,2,seq,32]

q = apply_rope(q, rope_cos, rope_sin)
k = apply_rope(k, rope_cos, rope_sin)

k = repeat_kv(k, self.n_rep)  # [b,2,seq,32]→[b,8,seq,32]
v = repeat_kv(v, self.n_rep)  # [b,2,seq,32]→[b,8,seq,32]
```

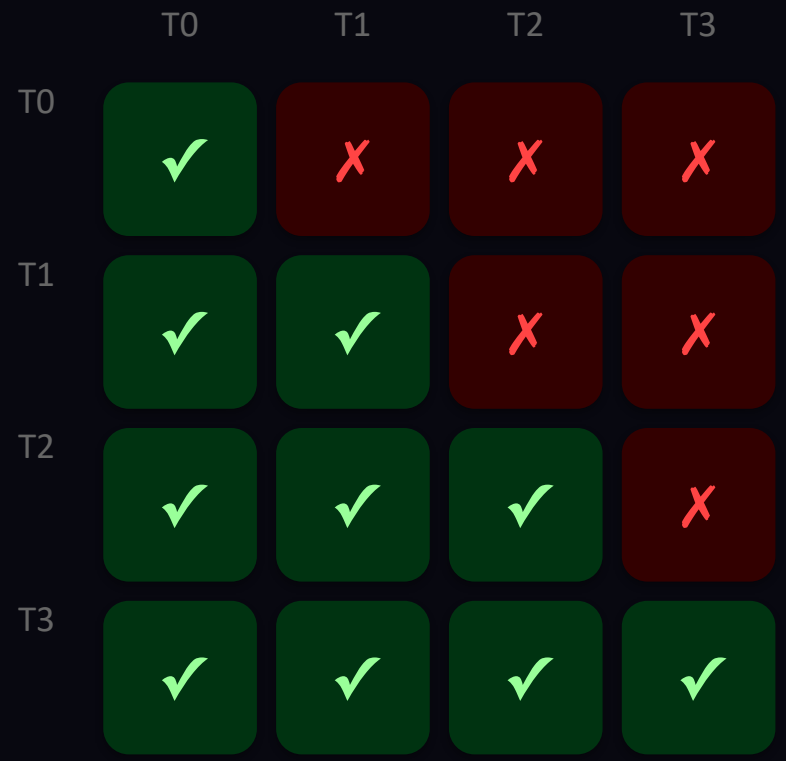
- Project → reshape into heads → apply RoPE → expand KV
- .view() splits flat vector into separate heads
- .transpose(1,2) swaps seq and heads for batched matmul
- RoPE on Q and K only — not V!

GQA — Forward: Attention Scores

```
scale = 1.0 / math.sqrt(self.head_dim)    # 1/√32

scores = (q @ k.transpose(-2, -1)) * scale # [b,8,seq,seq]

mask = torch.triu(
    torch.ones(seq, seq), diagonal=1
).bool()
scores = scores.masked_fill(mask, float("-inf"))

weights = F.softmax(scores, dim=-1)
weights = F.dropout(weights, p=0.2, training=self.training)
out = weights @ v                          # [b,8,seq,32]
```

- Q @ K^T — dot product of every Q with every K
- Scale by 1/√d prevents softmax saturation
- Causal mask: future positions → -∞ → softmax gives 0
- weights @ V — weighted sum = attention output

The Causal Mask



✓ can attend ✗ masked (-∞)

Each token can only attend to itself and previous tokens

GQA — Forward: Merge Heads & Output

```
out = out.transpose(1, 2).contiguous().view(b, seq, -1)
return self.o_proj(out)
```

- transpose + view: merge all 8 heads back into one vector
- [b, 8, seq, 32] → [b, seq, 256]
- o_proj: final linear projection 256 → 256
- .contiguous() needed before .view() after transpose

SECTION 4

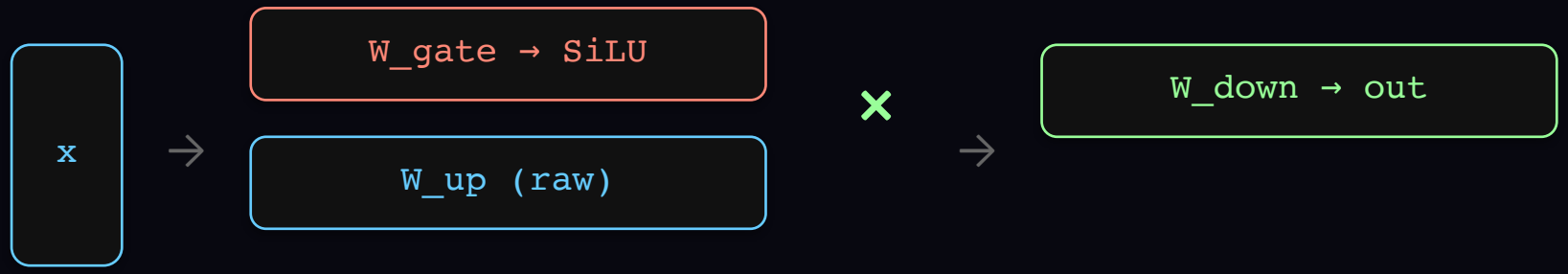
SwiGLU

Gated Feed-Forward Network

SwiGLU vs Old FFN

Old FFN: single path

SwiGLU: gated dual path



The gate LEARNS which dimensions to keep and which to suppress

SwiGLU — Code

```
class SwiGLU(nn.Module):
    def __init__(self, d_model, hidden_dim):
        self.w_gate = nn.Linear(d_model, hidden_dim)  # 256→680
        self.w_up   = nn.Linear(d_model, hidden_dim)  # 256→680
        self.w_down = nn.Linear(hidden_dim, d_model)  # 680→256

    def forward(self, x):
        gate = F.silu(self.w_gate(x))   # gate path + activation
        up   = self.w_up(x)             # value path (no activation)
        return F.dropout(
            self.w_down(gate * up), p=DROPOUT, training=self.training
        )
```

- Two parallel paths: gate (with SiLU) and up (raw)
- gate × up — element-wise multiply controls info flow
- SiLU(x) = x × sigmoid(x) — smooth version of ReLU
- w_down projects back: 680 → 256

SECTION 5

Assembly

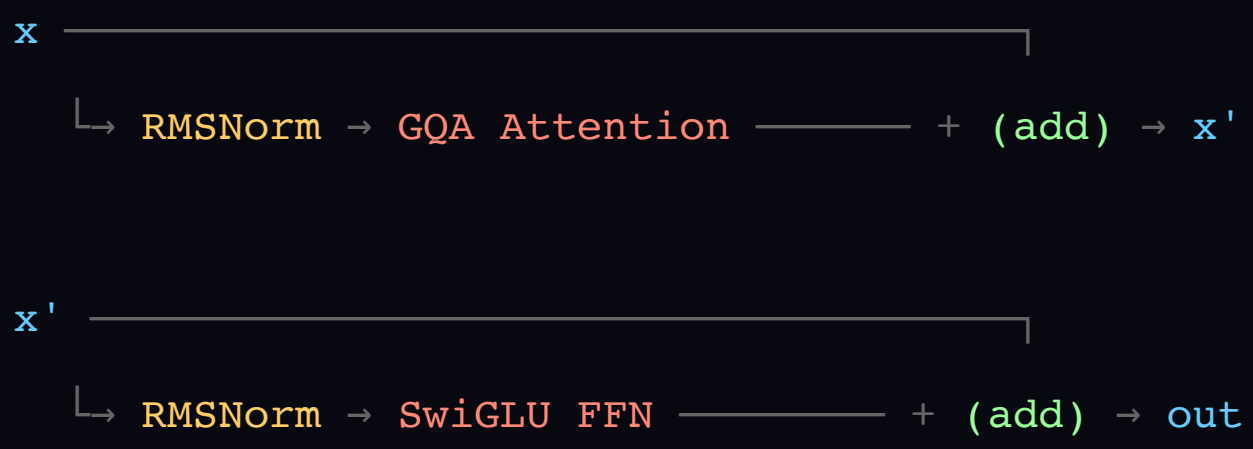
Putting It All Together

TransformerBlock — One Layer

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, n_kv_heads, ffn_hidden_dim):
        self.attn_norm = RMSNorm(d_model)
        self.attention = GroupedQueryAttention(d_model, n_heads, n_kv_heads)
        self.ffn_norm  = RMSNorm(d_model)
        self.ffn       = SwiGLU(d_model, ffn_hidden_dim)

    def forward(self, x, rope_cos, rope_sin):
        x = x + self.attention(self.attn_norm(x), rope_cos, rope_sin)
        x = x + self.ffn(self.ffn_norm(x))
        return x
```

- Pre-norm: normalize BEFORE each sub-layer
- Residual: x + sublayer(norm(x)) — learn the CHANGE
- Two sub-layers: attention (mix between tokens) + FFN (process each token)

Pre-Norm + Residual Pattern



Residual connections make deep networks trainable

MiniLLM — Full Model

```
class MiniLLM(nn.Module):
    def __init__(self, vocab_size, d_model, n_layers, ...):
        self.token_emb = nn.Embedding(vocab_size, d_model)  # 65→256

        self.layers = nn.ModuleList([
            TransformerBlock(...) for _ in range(n_layers)   # ×4
        ])

        self.final_norm = RMSNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size)       # 256→65

        # Weight tying!
        self.lm_head.weight = self.token_emb.weight
```

- No positional embedding — RoPE handles position inside attention
- Weight tying: embedding and output head share same weights
- 4 transformer blocks, each independently learned

MiniLLM — Forward Pass

```
def forward(self, idx, targets=None):
    x = self.token_emb(idx)                # [b, seq, 256]

    for layer in self.layers:               # 4 blocks
        x = layer(x, self.rope_cos, self.rope_sin)

    x = self.final_norm(x)                 # final RMSNorm
    logits = self.lm_head(x)               # [b, seq, 65]

    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.view(-1, logits.size(-1)),
            targets.view(-1)
        )
    return logits, loss
```

- Embed → 4 transformer blocks → norm → predict
- logits: a score for each of 65 characters at each position
- cross_entropy measures how wrong the predictions are

Model Configuration

`vocab_size`	**65**	characters in Shakespeare
`d_model`	**256**	embedding dimension
`n_layers`	**4**	transformer blocks
`n_heads`	**8**	query heads
`n_kv_heads`	**2**	KV heads (GQA 4:1)
`ffn_hidden_dim`	**680**	SwiGLU hidden size
`max_seq_len`	**256**	context window

Initial loss ≈ ln(65) ≈ 4.17 = random guessing → model is wired correctly!

SECTION 6

Training

The Training Loop

Training Hyperparameters

- BATCH_SIZE = 64 → processes 64 × 256 = 16,384 chars/step
- LEARNING_RATE = 3e-4 → standard for transformers
- MAX_STEPS = 3000 → ~15-20 min on T4 GPU
- AdamW optimizer — Adam + weight decay

The Training Loop

```
for step in range(MAX_STEPS):
    xb, yb = get_batch("train", BATCH_SIZE, CONTEXT_LEN)

    logits, loss = model(xb, yb)      # forward

    optimizer.zero_grad()             # clear old gradients
    loss.backward()                   # backprop
    torch.nn.utils.clip_grad_norm_(   # clip gradients
        model.parameters(), max_norm=1.0
    )
    optimizer.step()                  # update weights
```

- ① Get data ② Forward pass ③ Zero gradients
- ④ Backward (compute gradients) ⑤ Clip ⑥ Update
- Same 6-step dance from Class 2!
- Gradient clipping prevents exploding gradients in transformers

estimate_loss() — Evaluation

```
@torch.no_grad()
def estimate_loss():
    model.eval()            # disable dropout
    for split in ["train", "val"]:
        losses = []
        for _ in range(EVAL_STEPS):
            xb, yb = get_batch(split, BATCH_SIZE, CONTEXT_LEN)
            _, loss = model(xb, yb)
            losses.append(loss.item())
        out[split] = sum(losses) / len(losses)
    model.train()           # re-enable dropout
    return out
```

- @torch.no_grad() — no gradients needed, saves memory
- model.eval() disables dropout for fair evaluation
- Average over 20 batches for stable estimate
- Train vs Val gap → overfitting indicator

SECTION 7

Generation

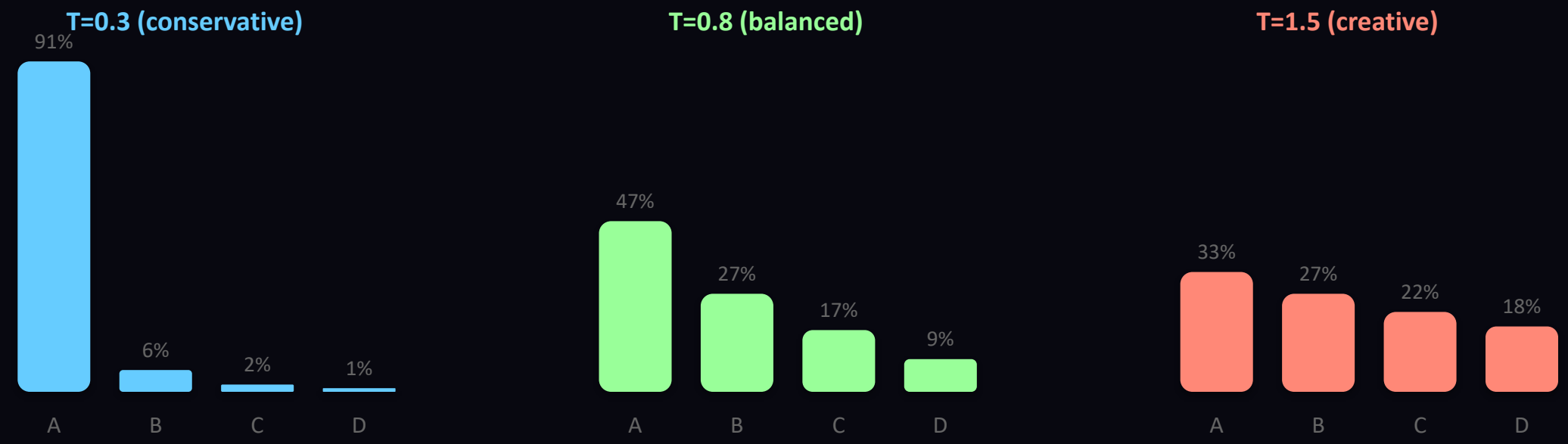
Autoregressive Text Generation

generate() — Autoregressive Sampling

```
@torch.no_grad()
def generate(model, prompt, max_new_tokens=500, temperature=0.8):
    model.eval()
    tokens = encode(prompt)
    tokens = torch.tensor(tokens, device=device).unsqueeze(0)

    for _ in range(max_new_tokens):
        context = tokens[:, -config["max_seq_len"]:]
        logits, _ = model(context)
        logits = logits[:, -1, :] / temperature
        probs = F.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)
        tokens = torch.cat([tokens, next_token], dim=1)

    return decode(tokens[0].tolist())
```

- Feed prompt → get logits → take LAST position only
- Divide by temperature → softmax → sample
- Append new token → repeat
- Sliding window: only last 256 tokens as context

Temperature — Controls Randomness



Same concept used in ChatGPT, Claude — tune creativity vs consistency

The Complete Architecture

This is not a toy — production LLMs use these exact same components, just bigger.